Research Methodology in Computer Science

WORKSHEET–02: UNIT II

Research Methodology in Computer Science

Dialectic, Arguments, Proofs, Deduction, Induction, Models and Evaluation

Name: Kushagra Singh **Roll No.:** 590012353

Section: B1[CCVT] **Date:** 15/9/2026

LEARN & REMEMBER

Purpose of this worksheet

This worksheet helps you understand and apply the major concepts of Unit II. Read the short explanation, complete the examples, and then attempt the practice questions.

CLASSROOM ACTIVITY

Unit II in one line

Claim → Argument → Challenge → Proof/Evidence → Model → Evaluation

UNIT II — LEARNING MAP

Tick each topic after completing it.

#	Topic	Done	Confidence
1	Dialectic of Research in CS	☑	1–5 5
2	Models of Argument	☑	1–5 4
3	Proof Methods	☑	1–5 5
4	Proof by Demonstration	☑	1–5 4
5	Empirical Proofs	☑	1–5 5
6	Mathematical Proofs	☑	1–5 4
7	Deduction and Induction	☑	1–5 4
8	Theoretical Models	☑	1–5 5
9	Algorithmic Approaches	☑	1–5 4
10	Software Engineering Approaches	☑	1–5 4
11	Mathematical Modelling	☑	1–5 5
12	Performance Evaluation	☑	1–5 4

PRACTICE ZONE

Quick warm-up

1. General → specific reasoning is called:

   Deduction

2. Specific observations → general rule is called:

   Induction

3. Reasoning toward the "best explanation" is called:

   Abduction

4. A claim becomes stronger when it is:

   tested against counter-arguments and supporting evidence

PART A — DIALECTIC OF RESEARCH

Using claims, counterclaims and evidence to refine research

LEARN & REMEMBER

Dialectic is structured reasoning through opposing or different views.
In Computer Science research, the process can be understood as:

$$\boxed{\text{Claim} \rightarrow \text{Challenge} \rightarrow \text{Evidence} \rightarrow \text{Refinement}}$$

It helps researchers:

- identify weaknesses in an idea,
- consider alternative viewpoints,
- reduce premature conclusions,
- refine vague research ideas,
- strengthen research arguments.

A1. Dialectical Process

1. Identify a claim or hypothesis.
2. Challenge it using counter-arguments or experiments.
3. Examine evidence and logic.
4. Refine the claim.
5. Arrive at a stronger explanation or solution.

PRACTICE ZONE

Practice
Claim:
"Greedy algorithms always produce optimal results."

1. What is the claim?
   Greedy algorithms always give the optimal solution.
2. Give a counterexample or situation where it may fail.
   0/1 knapsack: greedy by value/weight is not optimal.
3. Write a more accurate version of the claim.
   Greedy is optimal only when the problem has the greedy-choice property.
4. What evidence could support the revised claim?
   A proof of the greedy-choice property, plus counterexamples where it fails.

A2. Dialectic and Problem Formulation

Choose one:

AI in Education Smart Traffic Robotic Farming (IoT Security)

Initial idea	Detect intrusions in IoT networks using machine learning.
Possible claim	ML models can accurately flag IoT attacks in real time.
Counter-argument	IoT devices are resource-limited and traffic data is imbalanced.
Evidence required	Detection accuracy and latency on IoT-traffic datasets.
Refined research problem	Can a lightweight ML IDS detect IoT attacks with few false positives on-device?

PART B — MODELS OF ARGUMENT

Deductive, inductive and abductive reasoning

LEARN & REMEMBER

There are three important reasoning models:

	Deductive	Inductive	Abductive
Direction	General → Specific	Specific → General	Observation → Explanation
Conclusion	Certain if premises are true	Probable	Plausible
Typical CS use	Proof	ML / data analysis	Debugging

B1. Deductive Reasoning

General rule → specific case → conclusion.

Example

All Dijkstra results are optimal when its assumptions are satisfied.

Graph G satisfies those assumptions.

Therefore, the shortest-path result for G is optimal.

PRACTICE ZONE

Create your own deductive argument.

General rule:

All comparison-based sorts need >= n log n comparisons (worst case).

Specific case:

Merge sort is a comparison-based sort.

Conclusion:

So merge sort needs >= n log n comparisons in the worst case.

B2. Inductive Reasoning

Specific observations are used to form a general rule.

Example

A new sorting algorithm performs better than Quicksort on 100 test cases.

Possible inductive conclusion:

The new algorithm may perform better in general.

The conclusion is **not guaranteed**.

PRACTICE ZONE

A new ML model performs better on 8 datasets.

1. What are the observations?
   The model beat the baseline on all 8 datasets tested.
2. What general conclusion might be proposed?
   The model may generally outperform the baseline.
3. Why is the conclusion not certain?
   Only 8 datasets were tested; it may not generalize.

B3. Abductive Reasoning

Abduction searches for the most plausible explanation.

Example

Observation: A program crashes randomly.

Possible explanation: Memory leak.

PRACTICE ZONE

Observation: A web application suddenly becomes very slow.
Write three possible explanations.

1. Slow or unindexed database queries.
2. A server memory leak or CPU overload.
3. Network congestion or a sudden traffic spike.

Which explanation would you investigate first?
Check server resource metrics (CPU and memory) first.

PART C — PROOF METHODS

Demonstration, empirical and mathematical approaches

LEARN & REMEMBER

A proof or validation method provides evidence that a research claim is correct, feasible, reliable, or supported.

Method	Basis	Typical use	Main limitation
Demonstration	Examples / prototype	Early validation	Limited cases
Empirical	Experiments / data	Performance studies	Context-dependent
Mathematical	Logic / formal proof	Correctness / theory	Requires formal assumptions

C1. Identify the Method

Write D = Demonstration, E = Empirical, M = Mathematical.

1. Running a prototype on five sample inputs. **D**
2. Benchmarking an ML model on multiple datasets. **E**
3. Proving algorithm correctness using formal logic. **M**
4. Testing network latency under different workloads. **E**
5. Proving a theorem about computational complexity. **M**

C2. Choose the Validation Method

Research Claim	Method	Why?
A sorting algorithm is correct	M	Correctness needs a formal proof.
A new ML model performs better	E	Shown by experiments on datasets.
A robot prototype can follow a path	D	Demonstrated on a working prototype.
A cryptographic construction is secure under assumptions	M	Proved under formal assumptions.

PART D — PROOF BY DEMONSTRATION

Examples and prototypes as early evidence

LEARN & REMEMBER

Proof by demonstration means showing that something works using examples, test cases, or a prototype.
It is:

- easy to understand,
- useful for early research,
- useful for prototypes,
- useful for teaching,
- but not a universal proof.

D1. Example

Claim:

"This new sorting algorithm is efficient."

Suppose we run it on five sample arrays and all five executions are successful.

This demonstrates that it works for those tested examples.

It does **not** prove that it is efficient for every possible input.

PRACTICE ZONE

Consider:
"Our chatbot correctly handles all customer queries."
Design:

1. Test case 1: Ask a common FAQ (e.g., 'What are your opening hours?').
2. Test case 2: Ask an order-status query with a valid order ID.
3. Test case 3: Ask an ambiguous or misspelled question.
4. One corner case that might be missed: A multi-intent or out-of-scope query.

D2. Demonstration vs Proof

Complete:

Demonstration proves that the method works on the specific tested cases.

Demonstration cannot prove that it is correct for every possible input.

A better next step could be empirical testing on large data or a formal proof.

PART E — EMPIRICAL PROOFS

Evidence through observation, experimentation and measurement

LEARN & REMEMBER

Empirical validation uses real observations and experimental data.
A typical empirical study involves:

$$\text{Question} \rightarrow \text{Experiment} \rightarrow \text{Measurement} \rightarrow \text{Analysis} \rightarrow \text{Conclusion}$$

Common applications include:

- algorithm benchmarking,
- ML model evaluation,
- network performance,
- usability studies,
- system experiments.

E1. Example

Claim:

"Our new ML model performs better than the baseline."

Possible measurements:

- Accuracy
- Precision
- Recall
- F1-score
- Runtime

PRACTICE ZONE

Design an empirical experiment for:
"A new network protocol reduces latency."

Independent variable: Protocol used (new vs. baseline).

Dependent variable: Average end-to-end latency (ms).

Baseline: The existing standard protocol (e.g., TCP).

Dataset / workload: Simulated traffic at varying loads.

Metric: Mean latency and jitter.

Number of trials: 30 runs per condition.

Possible limitation: Simulation may not reflect real networks.

E2. Advantages and Limitations

Advantages

- Real-world evidence
- Practical performance
- Supports comparison
- Allows iterative improvement

Limitations

- Context-dependent
- Can be expensive
- External factors matter
- May not generalize

PART F — MATHEMATICAL PROOFS

Rigorous reasoning using logic and mathematics

LEARN & REMEMBER

Mathematical proof provides rigorous validation.
Important types:

1. Direct proof
2. Proof by contradiction
3. Proof by induction
4. Proof by contrapositive

Mathematical proof is widely used in algorithm correctness, complexity, cryptography, automata theory and formal systems.

F1. Direct Proof

A direct proof starts from known assumptions and logically derives the required result.

PRACTICE ZONE

Complete the structure.

Statement to prove:
The sum of two even integers is even.

Given / assumptions:
Let a = 2m and b = 2n for integers m, n.

Logical steps:
a + b = 2m + 2n = 2(m + n).

Conclusion:
a + b = 2(m + n), which is even.

F2. Proof by Contradiction

Basic idea:

Assume opposite → derive contradiction → original claim holds

Example in CS:

Impossibility results such as the Halting Problem can use contradiction-style reasoning.

PRACTICE ZONE

Fill the structure.

Claim:

sqrt(2) is irrational.

Opposite assumption:

Assume sqrt(2) = a/b in lowest terms.

Contradiction:

Then a^2 = 2b^2, so a and b are both even - not lowest terms.

Conclusion:

So sqrt(2) cannot be rational; it is irrational.

F3. Proof by Induction

Induction contains:

1. Base case
2. Inductive hypothesis
3. Inductive step
4. Conclusion

Classic result:

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

PRACTICE ZONE

Base case:

For $n = 1$:

$$1 = \frac{1(1+1)}{2}$$

Inductive hypothesis:

Assume:

$$1 + 2 + \cdots + k = \frac{k(k+1)}{2}$$

Inductive step:

$$1 + 2 + \cdots + k + (k+1)$$

$$= \frac{k(k+1)}{2} + (k+1)$$

Complete:

$$= \frac{(k+1)(\text{ k + 2 })}{2}$$

Conclusion:

By induction, 1 + 2 + ... + n = n(n + 1)/2 for all n >= 1.

PART G — CONTRAPOSITIVE

A useful form of implication-based reasoning

LEARN & REMEMBER

For an implication

$$P \Rightarrow Q$$

the contrapositive is

$$\neg Q \Rightarrow \neg P$$

An implication and its contrapositive are logically equivalent.

PRACTICE ZONE

Write the contrapositive of:
If an algorithm terminates, then it produces an output.
Original statement:
If an algorithm terminates, then it produces an output.

Contrapositive:
If it produces no output, then it does not terminate.

Explain why the two statements are logically related.
They are logically equivalent - true in exactly the same cases.

G1. Identify the Proof Type

Write:

Direct / Contradiction / Induction / Contrapositive

1. Assume the opposite and derive an impossible result. Contradiction
2. Prove the base case and then $k \to k+1$. Induction
3. Prove $\neg Q \to \neg P$ to establish $P \to Q$. Contrapositive
4. Start from known assumptions and derive the result directly. Direct

G2. Applications

Complete the table.

Algorithm correctness	Direct proof or induction.
Complexity analysis	Asymptotic bounds via induction.
Cryptography	Proof by contradiction / reduction.
Automata / formal languages	Induction and contrapositive.

PART H — DEDUCTION AND INDUCTION

Using evidence and logic to build CS knowledge

LEARN & REMEMBER

Deduction

General rule → Specific case

If the premises are true and the logic is valid, the conclusion follows.

Induction

Specific observations → General rule

The conclusion is probable rather than guaranteed.

H1. Compare

	Deduction	Induction	Abduction
Starts with	General rule	Observations	Observation
Ends with	Specific conclusion	Generalization	Best explanation
Certainty	High if premises true	Probable	Plausible

H2. Classify the Following

Write D, I or A.

1. A theorem is applied to a particular graph. D
2. A researcher finds the same trend in 100 experiments and proposes a general rule. I
3. A program crashes and the researcher suspects a memory leak. A
4. A formal proof establishes algorithm correctness. D

PRACTICE ZONE

Explain one answer

Choose any one of the above and explain why it is deductive, inductive or abductive.

Item 2 is inductive: a general rule is proposed from many specific observations.

PART I — THEORETICAL MODELS

Abstract models for understanding computational systems

LEARN & REMEMBER

Theoretical models provide abstract representations of computational systems.
Important examples include:

- Finite Automata
- Pushdown Automata
- Turing Machines
- Propositional and Predicate Logic
- Graphs and Trees
- Boolean Algebra
- Formal Grammars

They help researchers reason about computation without depending completely on a particular physical implementation.

I1. Match the Model

Recognize a formal language: Finite Automaton

Represent a network structure: Graph

Study computability: Turing Machine

Represent Boolean operations: Boolean Algebra

Reason about logical statements: Propositional / Predicate Logic

I2. Turing Machine

A Turing Machine is an abstract computational model used to study computability and decidability.

PRACTICE ZONE

Answer briefly.

1. Why can an abstract model be useful?
   It lets us reason about computation independently of hardware.

2. What is meant by computability?
   Whether a problem can be solved by an algorithm.

3. Give one research problem where a theoretical model would be useful.
   Proving a given language is not regular using automata theory.

I3. Advantages and Limitations

Advantages	Limitations
Clarity and precision	May ignore real-world constraints
Formal analysis	Can be highly abstract
Hardware-independent reasoning	May simplify complex systems

PART J — ALGORITHMIC APPROACHES

Designing, analysing and testing computational solutions

LEARN & REMEMBER

Algorithmic research commonly follows:

$$\text{Problem} \rightarrow \text{Algorithm} \rightarrow \text{Complexity} \rightarrow \text{Correctness} \rightarrow \text{Testing}$$

Examples include graph algorithms, sorting, searching, dynamic programming, greedy algorithms and approximation algorithms.

J1. Algorithm Research Canvas

Choose one problem.

Problem: Single-source shortest paths in a weighted graph.

Input: A weighted graph G and a source vertex s.

Output: Shortest-path distances from s to all vertices.

Algorithm idea: Dijkstra's algorithm with a min-priority queue.

Time complexity: O((V + E) log V).

Space complexity: O(V + E).

Correctness argument: Greedy choice with an inductive invariant on settled nodes.

Test cases: Positive-weight graphs; disconnected graphs.

J2. Think

Why should a researcher consider both theoretical complexity and practical execution time?

Big-O hides constants and hardware effects, so real runtime can differ; both matter.

COMMON TRAP

A successful implementation does not automatically prove optimality or correctness for every possible input.

PART K — SOFTWARE ENGINEERING APPROACHES

Applying structured development practices to research

LEARN & REMEMBER

Software engineering approaches provide systematic methods for developing and testing software systems.
Common process models include:

- Waterfall
- Agile
- Spiral

A simplified research software cycle is:

Requirements → Design → Implementation → Testing → Maintenance

K1. Research Prototype

Suppose you are developing a software prototype for a research project.

Requirements: Detect network intrusions with a low false-positive rate.

System design: Modular pipeline: capture -> features -> ML classifier.

Implementation: Python with scikit-learn and a packet parser.

Testing: Unit tests plus evaluation on a held-out dataset.

Future enhancement: Add real-time detection and a dashboard.

K2. Why Software Engineering Matters in Research

Give one reason for each:

Version control: Tracks changes and enables rollback/collaboration.

Testing: Catches bugs and confirms correct behaviour.

Documentation: Lets others understand and reuse the work.

Reproducibility: Lets others repeat and verify the results.

THINK LIKE A RESEARCHER

A research prototype may not become a commercial product. Why, nevertheless, should it be developed systematically?

So the prototype is reliable, reproducible, and easy for others to build on or verify.

PART L — MATHEMATICAL MODELLING

Representing real systems using mathematical relationships

LEARN & REMEMBER

Mathematical modelling converts a real-world system into a simplified mathematical representation. A basic process is:

System → Assumptions → Variables → Model → Analysis → Validation

Applications include network performance, reliability, queueing systems, graph problems and optimization.

L1. Build a Model

Choose one:

Network Cloud Robot AI System IoT System

System	A single-server packet-processing queue.
Research question	How does load affect average packet delay?
Assumption 1	Packet arrivals follow a Poisson process.
Assumption 2	Service times are exponential.
Variable 1	Arrival rate (lambda).
Variable 2	Service rate (mu).
Variable 3	Average number in system (L).
Relationship / equation	rho = lambda/mu; L = rho/(1 - rho).
How will you validate it?	Compare model predictions with simulation results.

L2. Simple Queueing Example

Suppose packets arrive at a server at rate λ packets/second and are processed at rate μ packets/second.

1. What does λ represent?
   The average packet arrival rate (packets/second).

2. What does μ represent?
   The average service (processing) rate (packets/second).

3. What would happen if the arrival rate becomes greater than the service rate?
   The queue grows without bound and the system becomes unstable.

PART M — PERFORMANCE ESTIMATION AND EVALUATION

Measuring efficiency, scalability and reliability

LEARN & REMEMBER

Performance evaluation assesses how effectively a system performs.
Important parameters include:

- **Time efficiency** — execution or response time
- **Space efficiency** — memory/storage
- **Scalability** — behaviour with increasing workload
- **Reliability** — consistency and failure behaviour

A basic evaluation cycle is:

Metric → Experiment → Measurement → Comparison → Interpretation

M1. Evaluation Plan

Research claim: Our model detects intrusions more accurately than the baseline.

Metric 1: Detection accuracy / F1-score.

Metric 2: False-positive rate.

Dataset / workload: CICIDS2017 network flows.

Baseline: A standard signature-based IDS.

Environment: Python on a single-GPU workstation.

Number of experiments: 10 runs with different random seeds.

Possible limitation: May not generalize to new attack types.

M2. Example

Suppose two ML models give:

Model	Accuracy	Precision	Recall	F1
A	94%	91%	88%	89%
B	92%	95%	86%	90%

1. Which model has higher accuracy? Model A (94%).
2. Which model has higher precision? Model B (95%).
3. Which model has higher F1-score? Model B (90%).
4. Can you declare one model universally "better" using only these values? Why?
   No - each wins on different metrics; the best choice depends on the goal.

PART N — PERFORMANCE TOOLS

Selecting appropriate tools for research evaluation

LEARN & REMEMBER

Examples of tools/categories used for performance studies include:

- Profilers
- Simulation tools
- Benchmarking suites
- Monitoring tools

Examples from the Unit II material include gprof, Valgrind, JProfiler, NS3, CloudSim, OMNeT++, SPEC CPU, MLPerf, Prometheus and Grafana.

PRACTICE ZONE

Match the requirement with an appropriate tool/category.

1. Measure CPU and memory behaviour of a program.
   Profiler (gprof / Valgrind / JProfiler).
2. Simulate network behaviour.
   Network simulator (NS3 / OMNeT++).
3. Compare standardized computing performance.
   Benchmark suite (SPEC CPU / MLPerf).
4. Monitor system metrics continuously.
   Monitoring tools (Prometheus / Grafana).

N1. Researcher's Performance Sheet

Experiment title:

Evaluating an ML-based network intrusion detector.

Hypothesis:

Feature selection lowers the false-positive rate without reducing recall.

Metric(s):

Precision, recall, and false-positive rate.

Observed result:

FP-rate fell from 5% to 1.8% at 95% recall.

Interpretation:

Feature selection cuts false alarms while keeping detection high.

Limitation:

Tested on one dataset; may not generalize.

PART O — INTEGRATED RESEARCH ACTIVITY

Apply multiple Unit II concepts to one research problem

CLASSROOM ACTIVITY

Scenario

A team proposes:

"Our AI-enabled smart irrigation system reduces water consumption while maintaining crop health."

You are the research-methods investigator.

Your task is to examine the claim scientifically.

O1. Dialectical Analysis

Original claim:

The AI irrigation system cuts water use while keeping crops healthy.

Counter-argument 1:

Sensor errors could mislead irrigation decisions.

Counter-argument 2:

Results may not hold across soil types or seasons.

Refined claim:

On the tested crops, the system cuts water use without harming yield.

O2. Argument Model

Which reasoning model would be appropriate?

Using observations to generalize performance — Inductive

Applying a known theorem to the system — Deductive

Finding the most likely cause of a sensor problem — Abductive

O3. Proof / Validation

Select appropriate methods:

Demonstration | (Empirical) | Mathematical

Explain your choice:

It is a performance claim, best tested by field experiments measuring water use and crop health.

O4. Algorithmic Approach

What algorithm could decide when irrigation should start?

A threshold policy (or decision tree) on soil-moisture level.

What would be its input?

Soil moisture, weather forecast, and crop type.

What would be its output?

Whether to start/stop irrigation (and for how long).

O5. Mathematical Model

Define three variables.

$x_1 =$ soil moisture level $\quad x_2 =$ water applied (litres) $\quad x_3 =$ crop water requirement

Write one assumption:

Crop water need is roughly constant within a growth stage.

Write a simple relationship/model:

Irrigate if x1 < threshold; amount = max(0, x3 - x1).

O6. Performance Evaluation

Metric 1	Water consumption (litres).
Metric 2	Crop-health index / yield.
Baseline	Fixed-schedule irrigation.
Experiment condition	Same crop and field over one season.
Expected result	Lower water use with similar crop health.

PART P — MINI RESEARCH DESIGN

Create your own small Computer Science research study

Research Title

Reducing False Positives in ML-Based Network Intrusion Detection.

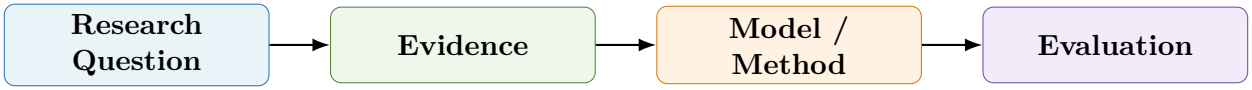
Research problem	ML-based IDS raise too many false alarms.
Research question	Can feature selection cut false positives without lowering recall?
Claim / hypothesis	Feature selection lowers the FP-rate at fixed recall.
Counter-argument	Fewer features might miss rare attacks.
Reasoning model	Inductive (empirical generalization).
Validation / proof method	Empirical evaluation on datasets.
Theoretical model	Feature space and classifier decision boundary.
Algorithmic approach	Random Forest with feature selection.
Mathematical model	FP-rate = FP / (FP + TN).
Performance metric	Precision, recall, and FP-rate.
Baseline	Full-feature classifier / signature IDS.
Expected contribution	A lighter IDS with fewer false alarms.

THINK LIKE A RESEARCHER

Researcher's check

Before accepting your own claim, ask:

1. What could prove me wrong?
2. What evidence do I need?
3. Which assumptions am I making?
4. Can another researcher reproduce my experiment?
5. Does my evidence actually support my conclusion?



UNIT II COMPLETE

Think critically. Question assumptions. Use evidence.
Build arguments. Validate claims. Evaluate results.

Research Methodology in Computer Science
School of Computer Science, UPES